

Final Project: AFL++ Crash Corpus Analysis

Student ID: 513558007

Target: Fuzzgoat JSON parser

Scope: replay all GitHub crash samples with AddressSanitizer

Repository: 2205jg/513558007-Fuzzing-Study

Coverage-guided fuzzing + ASan crash triage

Final Scope / 分析範圍

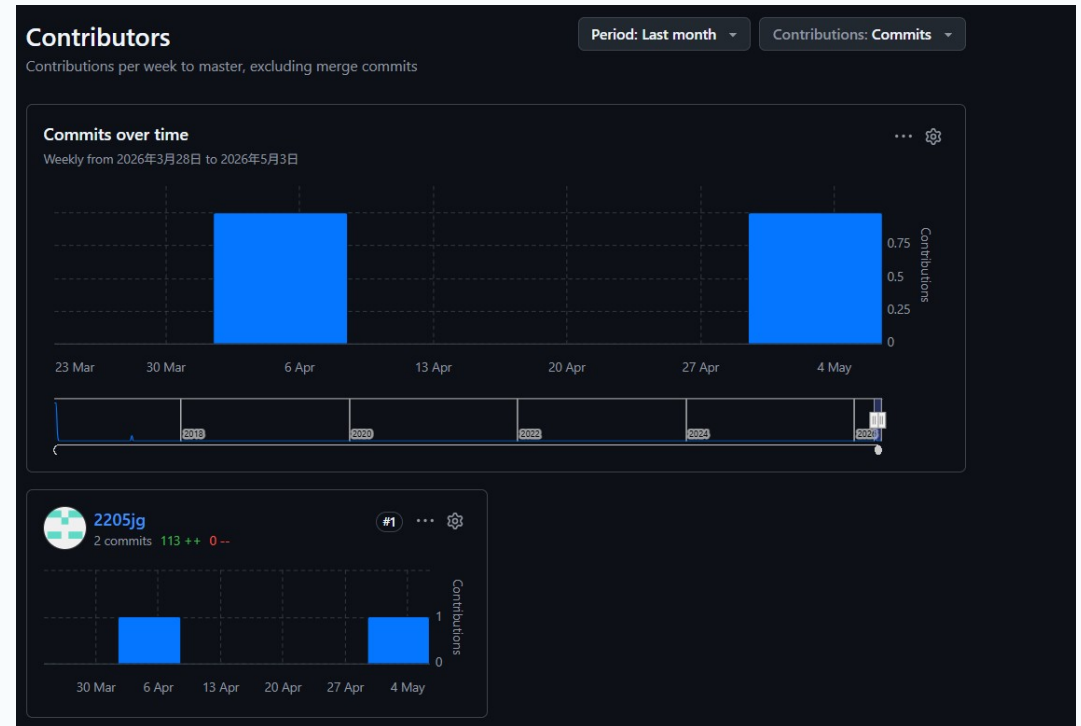
- Repository contains 65 AFL++ crash samples
- No samples were excluded, including id:000000 and id:000003
- Goal: classify all crashes and explain representative root causes
- Style: formal submission with technical evidence

Replay Methodology

- Clone repository into WSL/Linux to preserve AFL++ colon filenames
- Run fuzzgoat_ASAN against every out/default/crashes/id:* file
- Parse ASan type, signal, exit code, payload size and SHA-256
- Use addr2line to map binary offsets to source lines

GitHub Activity

- 2 project commits by 2205jg
- README records AFL++ / ASan setup progress
- Latest commit uploads crash corpus and fuzzgoat_ASAN binary
- Crash corpus became reproducible evidence, not just screenshots



Fuzzing Result Snapshot

- Original fuzzing run: about 1 day 2 hours 42 minutes
- Total executions: about 239M
- Saved crashes shown by AFL++: 65
- Final replay confirmed all 65 samples are executable evidence

```
root@HW: ~/513558007-Fuzzing-Study
┌─ process timing ─┴─┐
│ run time : 1 days, 2 hrs, 42 min, 33 sec
│ last new find : 0 days, 0 hrs, 37 min, 53 sec
│ last saved crash : 0 days, 3 hrs, 1 min, 3 sec
│ last saved hang : none seen yet
└─┘
┌─ cycle progress ─┴─┐
│ now processing : 264.7039 (30.7%)
│ runs timed out : 0 (0.00%)
└─┘
┌─ stage progress ─┴─┐
│ now trying : havoc
│ stage execs : 184/300 (61.33%)
│ total execs : 239M
│ exec speed : 2318/sec
└─┘
┌─ fuzzing strategy yields ─┴─┐
│ bit flips : 1/424, 0/419, 0/409
│ byte flips : 0/53, 0/48, 1/38
│ arithmetics : 5/3582, 0/5672, 0/4480
│ known ints : 2/438, 0/1733, 0/2068
│ dictionary : 0/0, 0/0, 0/0, 0/0
│ havoc/splice : 901/239M, 0/0
│ py/custom/rq : unused, unused, unused, unused
│ trim/eff : 9.95%/200k, 83.02%
└─┘
┌─ strategy: exploit ─┴─┐
│ state: in progress
└─┘
┌─ overall results ─┴─┐
│ cycles done : 1325
│ corpus count : 861
│ saved crashes : 65
│ saved hangs : 0
└─┘
┌─ map coverage ─┴─┐
│ map density : 11.40% / 76.94%
│ count coverage : 7.56 bits/tuple
└─┘
┌─ findings in depth ─┴─┐
│ favored items : 120 (13.94%)
│ new edges on : 162 (18.82%)
│ total crashes : 159k (65 saved)
│ total tmouts : 35 (0 saved)
└─┘
┌─ item geometry ─┴─┐
│ levels : 31
│ pending : 0
│ pend fav : 0
│ own finds : 860
│ imported : 0
│ stability : 100.00%
└─┘
[cpu:300%]
```

Classification Result

- heap-buffer-overflow: 39 samples
- heap-use-after-free: 16 samples
- invalid-free / bad-free: 9 samples
- segmentation-fault: 1 sample
- Unique stack groups: 5

Root Cause 1: Invalid Free

- Representative: id:000000
- ASan: attempting free on address which was not malloc()-ed
- Source: default_free at fuzzgoat.c:85
- Path also reaches main.c:166 and new_value at fuzzgoat.c:99

```
./fuzzgoat_ASAN out/default/crashes/id:000000*  
""  
-----  
string:  
=====1303411==ERROR: AddressSanitizer: attempting free on address which was not ma  
lloc()-ed: 0x50200000002f in thread T0  
#0 0x61a25059cffa in free (/src/fuzzgoat_ASAN+0xccffa) (BuildId: e9c7520006  
37e5054a2d38b8a89a1c1d6b08c082)  
#1 0x61a2505e64c0 in default_free /src/fuzzgoat.c:85:4  
#2 0x61a2505e64c0 in json_value_free_ex /src/fuzzgoat.c  
#3 0x61a2505e64c0 in json_value_free /src/fuzzgoat.c:1080:4  
#4 0x61a2505dea90 in main /src/main.c:166:9  
#5 0x7d1145b0f1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9f  
d827446c24067541ac5390e6f527fb5947bb)  
#6 0x7d1145b0f28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2  
a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)  
#7 0x61a2504fd464 in _start (/src/fuzzgoat_ASAN+0x2d464) (BuildId: e9c75200  
0637e5054a2d38b8a89a1c1d6b08c082)  
  
0x50200000002f is located 1 bytes before 1-byte region [0x502000000030,0x502000  
000031)
```

Root Cause 2: Heap Buffer Overflow

- Representative: id:000003
- ASan: heap-buffer-overflow
- Source: main.c:150, allocation around main.c:129
- Cause: file buffer is printed as a C string without NUL termination

```
[AFL++ 82b4749bb337] /AFLplusplus # cd /src
./fuzzgoat_ASAN out/default/crashes/id:000003*
=====
==1303413==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000001a at pc 0x55d1ff6d4a4b bp 0x7ffc251bb030 sp 0x7ffc251ba800
READ of size 11 at 0x502000000001a thread T0
#0 0x55d1ff6d4a4a in puts (/src/fuzzgoat_ASAN+0x4da4a) (BuildId: e9c752000637e5054a2d38b8a89a1c1d6b08c082)
#1 0x55d1ff795a36 in main /src/main.c:150:9
#2 0x76388526a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x76388526a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#4 0x55d1ff6b4464 in _start (/src/fuzzgoat_ASAN+0x2d464) (BuildId: e9c752000637e5054a2d38b8a89a1c1d6b08c082)

0x502000000001a is located 0 bytes after 10-byte region [0x5020000000010,0x502000000001a)
allocated by thread T0 here:
#0 0x55d1ff754293 in malloc (/src/fuzzgoat_ASAN+0xcd293) (BuildId: e9c752000637e5054a2d38b8a89a1c1d6b08c082)
#1 0x55d1ff7959de in main /src/main.c:129:32
#2 0x76388526a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
```


Other Stack Groups

- heap-use-after-free: 16 samples, top source json_parse_ex at fuzzgoat.c:643
- single extra heap-buffer-overflow: id:000016 at fuzzgoat.c:258
- segmentation-fault: id:000027 at json_value_free / fuzzgoat.c:224
- Many AFL++ inputs map to the same few root causes

Conclusion / 結論

- All GitHub crash samples were replayed and classified
- Final evidence files: crash_analysis.json and crash_analysis.csv
- The corpus collapses into 4 vulnerability classes and 5 source-level stack groups
- Next step: minimize representative inputs, patch root causes, and re-run regression tests